

SSRA: Scale-Shared Recursive Attention — Design and Complexity Analysis

Technical note (stage 1 of 2). **Author:** Daniel Sopov (SopovAi). ORCID: [0009-0004-8584-5156](https://orcid.org/0009-0004-8584-5156). **Status:** v1.1, 2026-07-20 — approved by the author. Changes from v1.0 (2026-06-11): an AI-assistance disclosure section, a decode-retention erratum (§2.6), and a stage-2 follow-up pointer; design and complexity class unchanged. **DOI:** [10.5281/zenodo.21462145](https://doi.org/10.5281/zenodo.21462145) (this version); v1.0: [10.5281/zenodo.20647034](https://doi.org/10.5281/zenodo.20647034). **Versioning:** This note fixes the design and its complexity analysis *before* any training run. The stage-2 paper with experimental results followed the evaluation plan in §8 and cites this note as the prior version: DOI [10.5281/zenodo.21439493](https://doi.org/10.5281/zenodo.21439493); see “Status of empirical follow-up” below. **License:** text CC BY 4.0; the reference implementation (to be released with the stage-2 paper) Apache-2.0.

Abstract

We describe SSRA (Scale-Shared Recursive Attention), a causal language-modeling block in which a single weight-shared rule — one softmax attention block Attn_θ and one learned pooling operator Pool_φ per layer — generates the entire scale hierarchy of the sequence. The same (θ, φ) is applied at every level of a binary tree built over the sequence, and the same θ performs the token-level read-out: a token is treated as a level-0 node. Two further properties complete the mechanism: learned, discrete-slot compression at every node (m summary vectors per node, via latent-query attention or hard top- m selection), and fully bidirectional attention inside nodes that remains legal in a causal LM because causality is enforced structurally — a node becomes consumable only after its span is complete — rather than by masking. Training cost is $\Theta(N \cdot (w + m \log N) \cdot d)$ per layer and the autoregressive decoding state is $O(m \cdot d \log N)$ per layer; this is the same complexity class as Log-Linear Attention, and we explicitly do not claim a better class. The contribution of this note is the design itself, its causality proof, its derived complexity, and a pre-registered falsification plan: two ablation axes (cross-scale weight sharing on/off; pooling operator family) and a multi-needle retrieval suite decide, at matched compute, whether the mechanism earns its place. Either outcome — positive or negative — answers the underlying hypotheses and is intended for publication.

1. Motivation

Natural language exhibits self-similar, long-range-dependent statistics: Alabdulmohsin et al. estimate a Hurst parameter of $H \approx 0.7$ for language and connect this fractal structure to next-token prediction [1]. Standard Transformers do not encode this property architecturally: every layer attends over a flat token sequence, and hierarchical variants in the literature typically introduce *per-level* parameters over a small, fixed number of levels.

SSRA tests a sharper inductive bias: **if language is self-similar, one rule should suffice at every scale**. Concretely, a single attention block and a single learned compressor per layer process pairs of tokens, pairs of summaries of pairs, and so on up a binary tree — and the very same attention block performs the final token-level read-out. The hypotheses this design operationalizes are:

- **H1 (scale-shared rule):** cross-scale weight sharing is an inductive bias matched to the self-similarity of language [1], yielding better length extrapolation at equal parameter budget than a flat baseline.
- **H2 (learned node compression):** m -slot learned compression at tree nodes preserves retrieval ability (measured by needle-in-a-haystack tests) at $\Theta(N \cdot (w + m \log N) \cdot d)$ training cost.

Both hypotheses are falsifiable by the pre-registered ablations in §8; the failure modes are spelled out in §6.

2. Mechanism overview

SSRA replaces the attention sublayer of a standard pre-norm Transformer block:

```
x ← x + SSRA_mix_i(LN1(x))      # this note
x ← x + FFN_i(LN2(x))          # standard MLP; not used inside tree nodes
```

Each layer i owns one attention parameter set θ_i (W_Q, W_K, W_V, W_O ; h heads), one pooling parameter set φ_i , and a learned level-embedding table e_ℓ . All three are **shared across all tree levels, all nodes, and the read-out within that layer**; nothing is shared across layers in this version. Each layer builds its own tree over its own current activations; summaries are per-layer activations, not persistent state.

Notation: sequence length N ; model width d ; tree level ℓ (tokens are level-0 nodes); node u at level ℓ spans 2^ℓ consecutive tokens; summary budget $m = 16$ (default); per-level summary count $s_\ell = \min(2^\ell, m)$; local read-out window $w = 64$ (default); tree arity $k = 2$ (default).

2.1 Up-pass (tree construction)

```

leaf u = token t:           S_u = z_t           # s_0 = 1
internal node u (children c1, c2, level  $\ell \geq 1$ ):
    X_u = concat(S_c1, S_c2)           #  $\leq 2m$  input slots
     $\tilde{X}_u = X_u + e_\ell$            # level coordinate as input
    H_u =  $\tilde{X}_u + \text{Attn}_\theta(\text{LN\_node}(\tilde{X}_u))$  # pre-norm residual;
                                           # bidirectional over slots;
                                           # relative slot-RoPE (positions 1.. $n_{in}$ )
                                           # identity while  $s_\ell = n_{in}$ 

    S_u = Pool_ $\psi$ (H_u,  $s_\ell$ )

```

Pre-norm residual paths and a LayerNorm in every node are hard requirements: without an identity path, $\log N$ successive rewrites destroy both verbatim signal transport and gradient flow. With fixed $m = 2^a$, levels $\ell \leq a$ are lossless (Pool = identity); the first lossy compression occurs at level $a+1$ ($32 \rightarrow 16$ slots for $m = 16$). For ragged N , a node is materialized iff its span lies inside $[1, N]$; no padding, no partial nodes. All nodes of a level are processed as one batched tensor (level-wise batching); recursion in the host language is prohibited in the training path.

2.2 Pooling operators Pool_ ψ

Three operators sit behind one interface (one operator per run, same operator at every lossy level):

operator	role	definition (sketch)	anchor
P1 latent-query attention pool	default	s_ℓ learned latent queries cross-attend to the node's slots, reusing the layer's θ projections; ψ adds only the queries and one LN	PMA / Set Transformer [16], Perceiver [17]
P2 strided pairwise merge	control	learned linear merge of adjacent slot pairs ($2 \rightarrow 1$)	Hourglass-style shortening [9]; token merging [19]
P3 learned top- $(s_\ell-1)$ selection + context residual	challenger	linear scorer; top slots copied verbatim (exact content pass-through), one extra slot is a temperature-softmax summary of the non-selected slots; straight-through or Gumbel-top-k gradients in training only; inference is hard and deterministic	natively trainable selection [6]

P1's latent queries are shared across **all** nodes and **all** scales of the tree — the maximal form of the scale-sharing bet, and (to our knowledge) a new use of PMA-style pooling. P2 isolates the question of whether selection must live in the pooling operator at all, or whether the shared Attn_θ can do the selection on its own. P3 trades smoothness for verbatim transport and is subject to a pre-registered stability gate (§8). A hybrid (k_{sel} verbatim slots + remaining latent-query slots) is a prepared fallback challenger.

2.3 Positional scheme: shared geometry, coordinate as input

location	treatment
inside a node	relative slot-RoPE over slot indices 1.. n_{in} — <i>identical geometry at every level</i> , which is exactly what makes the rule scale-transportable (the H1 bet); left/right child encoded by slot order
node input	learned level embedding e_ℓ added to every slot (initialized at zero; ablation OFF tests pure scale equivariance)
read-out, window keys	RoPE at absolute token positions (relative offsets $\leq w$ by construction)
read-out, summary keys	no rotary phase (NoPE) + e_ℓ added to keys only — the coordinate modulates addressing, not content

In the Fenwick decomposition (§2.5) the level of a block grows with its distance from the query, so e_ℓ doubles as a log-scaled distance code: recent context is addressed finely, old context coarsely. Mixing rotated token keys with special non-rotated keys in a single softmax follows the attention-sink / memory-token precedent [20], with one honest caveat: [20] assigns its persistent keys positions *within the cache*, i.e. the directly preceded treatment is a fixed virtual position rather than pure NoPE. Pure-NoPE summary keys are therefore a hypothesis checked in the first implementation milestone, with a fixed-virtual-position fallback specified in the design.

2.4 Causality without masks inside nodes

Three mechanisms, no causal mask inside nodes:

1. **Local window:** the read-out query at position t attends to tokens $[\max(1, t-w), t]$ only.
2. **Structural gating:** a node u is consumable by the read-out only once $\text{span}(u) \subseteq [1, t-w-1]$, and by its parent only when its sibling's span is also complete. Causality is an availability property of summaries, not a mask.
3. **Bidirectional attention inside nodes is legal.** Proof sketch: a summary S_u can influence the logit at position t only through an ancestor-or-self a that the read-out at t consumes, which requires $\text{span}(u) \subseteq \text{span}(a) \subseteq [1, t-w-1]$; hence every token inside u precedes t , and no information flows from positions $\geq t$ into the prediction at t . ■ The enumeration of paths is exhaustive: the only cross-node operation in the up-pass is parent composition — attention, normalization, and pooling all act within a single node — so the dataflow graph has no edges besides child-to-parent composition and the structurally gated read-out.

This buys an expressivity asymmetry over a flat causal Transformer: *within* a completed span, attention is unrestricted in both directions, because every consumer of that span sits strictly after it. A lesson encoded from an earlier prototype of this project: a decreasing training loss is *not* evidence of causal correctness (target leakage produces the same curve); only explicit tests are. Two are pre-registered: a **shift test** (perturbing token t must not change any logit at positions $< t$) and a **completion test** (incremental decoding must reproduce the full batched forward bit-for-bit within tolerance at every position).

2.5 Read-out (Fenwick prefix decomposition)

For query position t with prefix budget $p = t - w - 1$, the prefix $[1, p]$ is decomposed by the standard binary-indexed-tree (Fenwick) rule into $\leq \lceil \log_2 p \rceil + 1$ disjoint blocks, each of which is exactly a tree node (block ends at a multiple of its size; always materialized). The key/value set for position t is

```
K_t = { z_j : j ∈ [max(1, t-w), t] } ∪ { rows of S_u : u ∈ Fenwick(t-w-1) }
y_t = Attn_θ(q = z_t, kv = K_t)           # ONE softmax over heterogeneous keys
```

so $|K_t| \leq (w+1) + m \cdot (\lceil \log_2 t \rceil + 1)$, and window + Fenwick blocks cover $[1, t]$ disjointly and completely. The read-out **reuses θ** — the same attention block as in the nodes; a token is a level-0 node. Per-block capacity is m vectors *per block*, i.e. the macro context available to a query scales as $m \cdot \log t$ vectors, not a single m -vector bottleneck.

The Fenwick prefix structure itself is taken from Log-Linear Attention [2]; we claim no novelty for it (§6).

2.6 Autoregressive decoding

Per layer, the decoder maintains a ring buffer of the last $w+1$ token states and a node store governed by an explicit retention rule: at time t , retain node u iff $u \in \text{Frontier}(t) = \text{fenwick_blocks}(t)$ (needed to build future parents) **or** $u \in \text{Fenwick}(t-w-1)$ (needed by the read-out now). The naive rule “free children when the parent forms” is incorrect: the read-out frontier lags the tree frontier by $w+1$ positions, so children remain consumable for $w+1$ steps after their parent completes. Both sets have $\leq \lceil \log_2 t \rceil + 1$ members, giving per-layer state $\leq (w+1) \cdot d + 2 \cdot m \cdot d \cdot (\lceil \log_2 N \rceil + 1) = O(m \cdot d \cdot \log N)$. Amortized, one internal node completes per generated token; the worst case at $t = 2^k$ is a chain of k sequential completions — an $O(\log N)$ latency spike, accepted for the proof of concept and documented as a limitation.

Erratum (v1.1). This section states the retention rule in pointwise form — retain u iff $u \in \text{Frontier}(t) \cup \text{Fenwick}(t-w-1)$ — and, with §3, gives the decode-state bound with an explicit constant 2 on the log term. The pointwise rule is incorrect: $u \in \text{fenwick_blocks}(p)$ holds exactly for $p \in [e, e + 2^\ell - 1]$, where $e = \text{end}(u)$, so for levels with $2^\ell \leq w$ the Frontier interval and the read-out interval are disjoint, and the rule evicts u inside the gap $[e + 2^\ell, e + w]$ although the read-out requires it again at $t = e + w + 1$ — at $w = 64$ this affects every level $\ell \leq 6$, including level-0 blocks one step after leaving the local window. The corrected rule is the interval closure: retain u iff $t \in [e, e + 2^\ell + w]$. The fault was found during the first implementation milestone, after v1.0 of this note, and its correction is certified by the completion test of §2.4. The constant-2 form of the decode-state bound held only for levels with $2^\ell > w$; the corrected explicit bound is $O((m \cdot \log N + w \cdot \log m) \cdot d)$, $\approx 1.5 \times$ the v1.0 estimate at $m = 16$, $w = 64$, $N = 8k$ by direct count. The headline decoding class $O(m \cdot d \cdot \log N)$ at fixed w , m is unaffected (the w -dependent term is constant in N). This erratum mirrors §2.8 of the stage-2 record (DOI 10.5281/zenodo.21439493), which first published the correction; the v1.0 file of this note remains unmodified under its version DOI for provenance.

3. Complexity (derived; per layer, $\times L$ for the model)

Score-operation accounting, binary tree, pass-through schedule $s_\ell = \min(2^\ell, m)$:

quantity	bound	derivation sketch
up-pass, training	$\Theta(N \cdot m \cdot d)$	below the lossy threshold Σ_ℓ $c \cdot N \cdot 2^\ell \cdot d < 2cNmd$; above it $\sim N/m$ nodes at $4c \cdot m^2 \cdot d$ each $\Rightarrow < 4cNmd$
read-out, training	$\Theta(N \cdot (w + m \cdot \log N) \cdot d)$	position t sees $\leq (w+1) + m(\lceil \log_2 t \rceil + 1)$ keys; sum over t read-out dominates; with w, m, d constant this is $\Theta(N \log N)$ — the same class as Log-Linear Attention
training total	$\Theta(N \cdot (w + m \cdot \log N) \cdot d)$	[2], not better
activation memory (tree)	$O(N \cdot d \cdot \log m)$	$\approx 5\times$ flat token activations at $m = 16$; score matrices are $N \cdot (w + m \cdot \log N)$ vs N^2 flat
decode state	$O(m \cdot d \cdot \log N)$	retention rule of §2.6, explicit constant 2 on the log term
decode compute / token	$O((w + m \cdot \log N + m^2) \cdot d)$ amortized	read-out + amortized one node completion
decode worst-case latency	$O(\log N)$ sequential completions at $t = 2^k$	documented limitation

Honest accounting. The QKVO projections add $\Theta(N \cdot d^2)$ terms. These are class-neutral, but at $d \approx 512$ and $N = 8k$ they are of the same order as the score operations. Consequently, any wall-clock advantage over a flat Transformer is an *empirical* question decided by measured throughput curves (§8), not by the asymptotics above. We do not claim a wall-clock advantage in this note.

For calibration: a flat Transformer trains at $\Theta(N^2 \cdot d)$ with $O(N \cdot d)$ decoding state; H-Transformer-1D reaches linear time and memory by hierarchical low-rank approximation [3]; Log-Linear Attention trains at $O(T \log T)$ with $O(\log T)$ state [2]; prefix-scannable models reach $O(\log N)$ state with softmax-like aggregation operators [23]. SSRA's claim is not a better complexity class than [2] or [23] — it is a different mechanism inside the same class, and §8 specifies the measurements that decide whether that mechanism is worth having.

4. Parameter inventory

Per layer, SSRA's overhead relative to a flat pre-norm Transformer at equal d, h, L is the level-embedding table, one node LayerNorm, and φ . At $d = 512, m = 16, N_{\max} = 32k$ this is $\approx 26k$ parameters per layer — well under 1 % of a layer. P1's φ is $m \cdot d + 2d$ (latent queries + a LayerNorm; the attention projections are reused from θ); P2's is $2d^2 + d$; P3's is $d + 1$. Parameter-matched comparisons with the flat baseline are therefore nearly free.

5. What this design deliberately is not (non-goals)

No top-down / coarse-to-fine output pass (a candidate follow-up, see §7). No content-based or learned segment boundaries — the hierarchy is strictly positional; learned boundaries are an orthogonal axis explored by dynamic token pooling [25] and H-Net [13]. No claim of a better complexity class than [2]. No KV-cache tricks, quantization, or kernel engineering beyond what the throughput measurement needs. No inference-time stochasticity of any kind.

6. Novelty thesis and delimitation

Claim. SSRA is a causal language-modeling block in which a single weight-shared rule — one softmax attention block Attn_θ and one learned pooling operator Pool_φ per layer — generates the entire scale hierarchy of the sequence. The same (θ, φ) applies at every level of a binary tree over the sequence, and the same θ performs the token-level read-out: a token is a level-0 node. Two properties complete the mechanism: (i) learned, discrete-slot compression at every node (m summary vectors via latent-query attention or hard top- m selection), and (ii) fully bidirectional attention inside nodes that remains legal in a causal LM because causality is enforced structurally — a node becomes consumable only after its span is complete — rather than by masking.

Delimitation. The prefix decomposition used by the read-out is the standard Fenwick (binary indexed tree) structure shared with Log-Linear Attention [2]; we claim no novelty for it. Relative to [2], the mechanism differs in all three

remaining components: softmax attention vs. linear-attention kernels; learned discrete-slot pooling vs. structural matrix state with λ -decay; and cross-scale parameter sharing that includes the read-out. Relative to hierarchical and multiresolution models [3, 8, 9], SSRA shares one rule across all levels instead of per-level parameters in fixed 2–3-level stacks; notably, Fast Multipole Attention learns its group-compression weights but learns them *per level* and aggregates linearly rather than with a shared attention block [4], and MRA attention uses a fixed wavelet family [5]. Relative to weight sharing in depth [10, 11], sharing here is across sequence scale. PMA/Perceiver-style pooling [16, 17] anchors the P1 operator; its use as the node compressor of a causal scale hierarchy with queries shared across all nodes and scales is, to our knowledge, new. The nearest combinations found in a verification pass and an arXiv novelty scan dated 2026-06-11 are delimited explicitly: GPST [21] shares a learned composition function over an induced *syntactic* tree inside a generative LM, but composes constituents to single vectors, keeps the composition model separate from the generative model, and makes no long-context or retrieval claim; MANO [22] shares convolution kernels and attention weights across scales, but for encoder-style vision/physics grids without causality, slot summaries, or a prefix read-out; prefix-scannable models [23] obtain softmax-like operators with $O(\log N)$ decoding state via prefix scans over chunk encodings, without cross-scale rule sharing or learned per-node slot compression; the Hierarchical Kernel Transformer [24] computes independent score matrices per resolution from trainable causal downsampling and fuses them, again without a shared rule or node summaries. No work among the surveyed set [1–25] simultaneously combines: cross-scale weight sharing $\times$ learned m-slot node compression $\times$ causal LM over text. (SSRA is also unrelated, beyond the name, to Fractal Generative Models for images [15].)

Falsifiability. The thesis is decided by two pre-registered ablation axes at matched compute: if cross-scale sharing does not help (ablation a, §8), SSRA degenerates into a slower relative of H-Transformer-1D [3]; if learned node compression loses retrievable detail (multi-needle suite, prediction P-B, §8), SSRA loses to Log-Linear Attention [2] on recall. Either outcome answers H1–H2 and is publishable.

7. Deferred design branch: top-down output path

A second output-path variant — a full top-down cascade in which coarse states condition finer ones — was designed and rejected for this version: at representative hyperparameters it is substantially more expensive to train, its macro state is stale by up to 2^ℓ tokens at level ℓ , and it routes the entire macro context through m vectors per node, a structural bottleneck on exactly the retrieval axis where the empirical contest with [2] is decided. It is retained as a named follow-up branch (SSRA-TD), motivated by a separate hypothesis (top-down conditioning improves long-generation consistency) that this proof of concept does not test.

8. Pre-registered evaluation plan

Verification before any training conclusion (milestone M1). Shift test (causality), completion test (incremental $\equiv$ batched forward at every position; certifies the Fenwick logic and the decode retention rule simultaneously), gradient-flow check across all parameter groups, and throughput/VRAM curves vs $N \in \{1k \dots 8k+\}$ against a flat baseline at equal d, h, L . Acceptance proposal for the throughput curve: log-log slope ≤ 1.5 over the range and strictly below the flat baseline’s slope. A stability micro-gate for P3: on a $\sim 10M$ -parameter smoke run, P3 must reach a loss within 5 % (relative) of P1 without divergence after standard stabilization (load-balance auxiliary loss, temperature annealing); failure demotes P3 to an appendix and promotes the P1 $\times$ P3 hybrid to challenger.

Training (M2). Two scales ($\approx 25M$ for tuning, $\approx 85M$ for the main comparison) on a public web-text sample, token budget $\approx 20\times$ parameters, training context 1k–8k. Gate: stable training and short-context perplexity within ± 5 % of the compute-matched flat baseline.

Evaluation (M3). Baselines: (a) compute-matched flat Transformer; (b) Log-Linear Attention / Gated DeltaNet from a public implementation [2]; (c) a MEGABYTE-style two-level model [8]. Metrics: perplexity vs context length (1k $\rightarrow$ 32k, beyond training length), per-position loss, needle-in-a-haystack and a **multi-needle suite** ($k \in \{1, 2, 4, 8, 16\}$ needles, distances from 128 to full context, exact-match recall), throughput and memory in training and decoding (including the worst-case completion spike), and pooling-collapse diagnostics (entropy of latent-query attention maps, per-query participation).

Pre-registered predictions. - **P-A:** with P2 (control), needle recall decays roughly with the number of compression levels between needle and query; P1/P3 stay approximately flat up to a capacity ceiling. - **P-B (sharpest discriminator):** k needles inside one subtree produce a recall cliff near $k \approx m$ for the learned operators (P3 near $m-1$; P1 earlier if query collapse occurs) and near $k \approx 1$ for P2. - **P-C:** latent-query collapse, if it occurs, is visible in training-time entropy/participation diagnostics before it is visible in task metrics.

Ablation axes (registered). (a) cross-scale weight sharing on/off (per-level θ copies) — the core H1 test; (b) Pool $_{\varphi} \in \{P1, P2, P3, \text{hybrid}\}$; (c) m fixed vs linearly growing schedule; (d) $w \in \{32, 64, 128\}$; (e) read-out parameters shared vs separate; (f) level embedding on/off; (g) arity $k \in \{2, 4\}$ and (h) the induced depth change. The $k = 2$ default is chosen

deliberately: it matches the Fenwick skeleton of [2] exactly, so the comparison isolates the mechanism rather than the tree structure.

Outcome policy. Success is defined as a clear, publishable answer to H1–H2 at matched compute — positive *or* negative. A measurable long-context advantage (perplexity tail or needle recall) at $\geq$ equal short-context perplexity yields a positive stage-2 paper; otherwise the stage-2 paper is an analytical negative result locating where and why the mechanism fails.

9. Limitations

Small scale only ($\leq \sim 85\text{M}$ parameters); a single text domain; no scaling laws. No experimental results exist at the time of this note — every empirical statement above is a pre-registered plan, not a finding. The top-down branch (SSRA-TD, §7) and content-based hierarchies are untested deferrals. The complexity statements count score operations; QKVO projection terms are of comparable magnitude at the planned widths, so wall-clock conclusions await measurement (§3). Decoding has an $O(\log N)$ worst-case latency spike at power-of-two positions. Activation memory of the tree is $\approx 5\times$ flat token activations at $m = 16$ (score-matrix memory is, however, far below the flat N^2 term at the planned lengths). Pure-NoPE summary keys in the read-out rest on an adjacent rather than exact precedent (§2.3) and carry a specified fallback.

Status of empirical follow-up (v1.1)

A stage-2 record, *SSRA: Scale-Shared Recursive Attention — Empirical Evaluation and Negative Results* (DOI [10.5281/zenodo.21439493](https://doi.org/10.5281/zenodo.21439493)), reports the pre-registered evaluation of §8 at $\approx 85\text{M}$ parameters. The gate of §8 — short-context perplexity within $\pm 5\%$ of the compute-matched flat baseline — was not met, so H1 is not supported at that scale and configuration, and the later baselines and ablations of §8 did not run, per the plan’s stop-loss. The stage-2 record reports this as a pre-registered negative result, together with an erratum to this note (its §2.8, mirrored in §2.6 above) and descriptive diagnostics of the trained models. It does not revise the design or the complexity derivations of this note, which are analytical.

AI Assistance Disclosure

This research was conducted as a human-directed AI collaboration. The research goal and the underlying inspiration (a fractal, tree-like structure over the sequence context as a path to more efficient attention) come from the author. The specific architecture design, its mathematical formulation, the complexity analysis, and the PyTorch implementation were generated by AI systems (Anthropic Claude, primarily through Claude Code) under the author’s direction. The author defined the research goal and requirements, reviewed and tested the generated work, ran the experiments and measurements, and takes responsibility for the published content. An automated verification suite (causality, equivalence and gradient-flow checks) and empirical measurements were used to validate the AI-generated components.

The prior-art review was likewise conducted by AI under the author’s direction. The text of this report was written by AI and reviewed by the author. This disclosure was added to this record’s Zenodo metadata on 9 July 2026; the original publication did not state the extent of AI involvement. Version 1.1 incorporates this section into the note file itself; the stage-2 record (DOI [10.5281/zenodo.21439493](https://doi.org/10.5281/zenodo.21439493)) carries an equivalent disclosure with refined execution details.

Version history

- **v1.0** (2026-06-11): stage-1 publication. DOI [10.5281/zenodo.20647034](https://doi.org/10.5281/zenodo.20647034).
- **v1.1** (2026-07-20): adds the AI Assistance Disclosure section, the §2.6 decode-retention erratum (mirroring §2.8 of the stage-2 record), and the “Status of empirical follow-up” section; header updated accordingly. Design and complexity class unchanged; the explicit decode-state constant of §2.6/§3 is corrected by the erratum.

References

All URLs retrieved 2026-06-09 or 2026-06-11.

1. Alabdulmohsin et al. *Fractal Patterns May Illuminate the Success of Next-Token Prediction*. NeurIPS 2024. <https://arxiv.org/abs/2402.01825>
2. Guo et al. *Log-Linear Attention*. ICLR 2026. <https://arxiv.org/abs/2506.04761>
3. Zhu, Soricut. *H-Transformer-1D: Fast One-Dimensional Hierarchical Attention for Sequences*. ACL-IJCNLP 2021. <https://arxiv.org/abs/2107.11906>
4. Kang, Tran, De Sterck. *Fast Multipole Attention: A Divide-and-Conquer Attention Mechanism for Long Sequences*. 2023. <https://arxiv.org/abs/2310.11960>

5. Zeng et al. *Multi Resolution Analysis (MRA) for Approximate Self-Attention*. ICML 2022. <https://arxiv.org/abs/2207.10284>
6. DeepSeek. *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*. ACL 2025. <https://arxiv.org/abs/2502.11089>
7. Munkhdalai et al. *Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention*. 2024. <https://arxiv.org/abs/2404.07143>
8. Yu et al. *MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers*. NeurIPS 2023. <https://arxiv.org/abs/2305.07185>
9. Nawrot et al. *Hierarchical Transformers Are More Efficient Language Models*. Findings of NAACL 2022. <https://arxiv.org/abs/2110.13711>
10. Dehghani et al. *Universal Transformers*. ICLR 2019. <https://arxiv.org/abs/1807.03819>
11. Bae et al. *Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation*. NeurIPS 2025. <https://arxiv.org/abs/2507.10524>
12. Hu et al. *R2D2: Recursive Transformer based on Differentiable Tree for Interpretable Hierarchical Language Modeling*. ACL-IJCNLP 2021. <https://arxiv.org/abs/2107.00967>
13. Hwang, Wang, Gu. *Dynamic Chunking for End-to-End Hierarchical Sequence Modeling*. 2025. <https://arxiv.org/abs/2507.07955>
14. Wang et al. *Hierarchical Reasoning Model*. 2025. <https://arxiv.org/abs/2506.21734>
15. Li et al. *Fractal Generative Models*. 2025. <https://arxiv.org/abs/2502.17437>
16. Lee et al. *Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks*. ICML 2019. <https://arxiv.org/abs/1810.00825>
17. Jaegle et al. *Perceiver: General Perception with Iterative Attention*. ICML 2021. <https://arxiv.org/abs/2103.03206>
18. Mu, Li, Goodman. *Learning to Compress Prompts with Gist Tokens*. NeurIPS 2023. <https://arxiv.org/abs/2304.08467>
19. Bolya et al. *Token Merging: Your ViT But Faster*. ICLR 2023. <https://arxiv.org/abs/2210.09461>
20. Xiao et al. *Efficient Streaming Language Models with Attention Sinks*. ICLR 2024. <https://arxiv.org/abs/2309.17453>
21. Hu, Ji, Zhu, Wu, Tu. *Generative Pretrained Structured Transformers: Unsupervised Syntactic Language Models at Scale*. 2024. <https://arxiv.org/abs/2403.08293>
22. Colagrande, Caillon, Feillet, Allauzen. *Linear Attention with Global Context: A Multipole Attention Mechanism for Vision and Physics (MANO)*. 2025. <https://arxiv.org/abs/2507.02748>
23. Yau et al. *Sequential-Parallel Duality in Prefix Scannable Models*. 2025. <https://arxiv.org/abs/2506.10918>
24. Cirrincione. *Hierarchical Kernel Transformer: Multi-Scale Attention with an Information-Theoretic Approximation Analysis*. 2026. <https://arxiv.org/abs/2604.08829>
25. Nawrot, Chorowski, Łańcucki, Ponti. *Efficient Transformers with Dynamic Token Pooling*. ACL 2023. <https://arxiv.org/abs/2211.09761>